

Computer Programming 1

Laboratory 05 (part 3)

fundamental data types – instructions – (basics) functions

ASCII encoding

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

Example 1: Pascal Triangle

Context: In the Pascal's Triangle, the numbers are presented according to a triangular structure by following a specific pattern. Various patterns can be observed in Pascal's Triangle, e.g., Diagonals, Horizontal Sum, Binomial Coefficients, etc.

Task: Implements a program for printing the numbers of the Pascal's Triangle with the Binomial Coefficients pattern. The program takes an integer value n as input, and it prints first n lines of the Pascal's triangle.

The value of c th entry (column) in the line number lin is computed as:

$$C \binom{lin}{c} = \frac{lin!}{(lin - c)! * c!}$$

Suggestion: both lin and c start from 0

Examples

Input: 5

Output:

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

Input: 8

Output:

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

1 5 10 10 5 1

1 6 15 20 15 6 1

1 7 21 35 35 21 7 1

Example 2: Type of Triangle

A triangle can be:

- Equilateral, if all three sides have the same length;
- Isosceles, if at least two sides have the same length (approximation);
- Scalene, if all sides have different lengths.

Task: Implements a program that recognize the type of triangle given three the length of the three sides in inputs from the user: a, b, and c.

Remember to check that all sides have length > 0 and that the sum of the lengths of any two sides must be greater than or equal to the length of the third side.

Examples

Input: 3 4 5
Output: scalene

Input: 3 4 0
Output: error

Input: 2 2 4
Output: isosceles

Input: 2 2 2
Output: equilateral

Implementation constraint (1): use an enumeration like: «enum TType {error,equilateral,isosceles,scalene,init};» where:

- init is used only for initialization
- error is used when a check detects wrong inputs
- equilateral, isosceles, scalene are used for the detected triangle

Example 2: Type of Triangle

A triangle can be:

- Equilateral, if all three sides have the same length;
- Isosceles, if at least two sides have the same length (approximation);
- Scalene, if all sides have different lengths.

Task: Implements a program that recognize the type of triangle given three the length of the three sides in inputs: a, b, and c.

Remember to check that all sides have length > 0 and that the sum of the lengths of any two sides must be greater than or equal to the length of the third side.

Examples

Input: 3 4 5
Output: scalene

Input: 3 4 0
Output: error

Input: 2 2 4
Output: isosceles

Input: 2 2 2
Output: equilateral

Implementation constraint (2): ask the user to enter the three sides of the triangle (a, b, c), use `cin.fail`, `cin.clean`, and `cin.ignore` to check the input stream after each read, and to clean it in case of error. In case of error does not close the program but re-ask the user to enter the value/s

Example 3: Basics with functions 1

Write a program that ask the user for two numbers as input, compare them and show the maximum.

Implementation constraint: Declare a function called `max_two` that compares the numbers and returns the maximum, while the main reads the inputs and writes the results to the standard output

Example 4: Basics with functions 2

Write a program that ask a number to the user as input, compute the sum of the numbers from 0 up to such a number and return it to the user.

Implementation constraint: Declare a function called `sumUpTp` that sums the numbers and returns the sum, while the main reads the input value and writes the results to the standard output

Example 5: Basics with functions 3

Write a program to print prime numbers between two numbers defined by the user as input of the program.

Implementation constraint: Declare a function called `isPrime` that gets two numbers as parameters and returns `true` if the number is a prime number, `false` otherwise. The function *main* reads the input values and writes the results to the standard output

Example 6: Basics with functions 4

Write a program to check if two given non-negative integers have the same last digit.

Sample Input:

- 123, 456
- 12, 512
- 7, 87
- 12, 45

Sample Output:

- 0
- 1
- 1
- 0

Implementation constraint: Declare a function called *check* to check if the last digits of two numbers have the same absolute value, while the main reads the input numbers and writes the results to the standard output